Olimpiada Națională de Informatică, Baraj pentru selecția lotului național de seniori, Ziua 1
Descrierea soluțiilor

Comisia științifică

March 29, 2026

Problema 1. Inspirație

Propusă de: Stud. Alexandru Lorintz, Universitatea "Babeş-Bolyai", Cluj-Napoca

Pentru un punct de query $Q(X, Y)$ și un punct de inspirație $A(x, y, v)$, distanța Manhattan dintre cele două este $D = |X - x| + |Y - y|$. Inspirația pe care Q o primește de la A este $\max(0, v - D)$. Din cauza modulelor din formula distanței, evaluarea directă este dificilă. Putem simplifica problema calculând independent contribuția din fiecare dintre cele 4 cadrane relative la punctul Q.

Analiza unui cadran

Să analizăm cadranul din stânga-jos față de Q: considerăm doar punctele de comisie care respectă $x \leq X$ și $y \leq Y$. Pentru acestea, distanța până la un punct (X, Y) de query devine $D = (X - x) + (Y - y)$. Inspirația primită va fi:

$$v - (X - x + Y - y) = (x + y + v) - (X + Y)$$

Pentru ca punctul de query să primească inspirație, trebuie ca această cantitate să fie strict pozitivă, adică:

$$x + y + v > X + Y$$

Dacă notăm „valoarea" punctului de inspirație $V_A = x + y + v$ și „valoarea" query-ului $V_Q = X + Y$, inspirația produsă este $V_A - V_Q$. Noi trebuie să calculăm $\sum(V_A - V_Q)$ pentru toate punctele comisiei care respectă simultan următoarele 3 condiții:

1. $V_A > V_Q$
2. $x \leq X$

3. $y \leq Y$

Pentru a număra și însuma eficient, putem renunța la menținerea simultană a condițiilor geometrice folosind Principiul Includerii și Excluderii. Ne propunem să calculăm rezultatul astfel:

- S_{total}: Suma $V_A - V_Q$ pentru **toate** punctele comisiei cu $V_A > V_Q$.
- S_x: Suma pentru punctele comisiei cu $V_A > V_Q$ care încalcă prima limitare ($x > X$).
- S_y: Suma pentru punctele comisiei cu $V_A > V_Q$ care încalcă a doua limitare ($y > Y$).
- S_{xy}: Suma pentru punctele comisiei care încalcă **ambele** limitări ($x > X$ și $y > Y$).

Damage-ul valid pe acest cadran va fi: $S_{total} - S_x - S_y + S_{xy}$.

Observația cheie: Pentru termenul S_{xy}, știm că $x > X$ și $y > Y$. Cum coordonatele sunt întregi, avem $x \geq X + 1$ și $y \geq Y + 1$. Astfel:

$$V_A = x + y + v \geq (X + 1) + (Y + 1) + v = X + Y + v + 2$$

Deoarece $v \geq 1$, rezultă clar că $V_A > X + Y + 3 > V_Q$. Așadar, condiția $V_A > V_Q$ este **implicit îndeplinită** atunci când $x > X$ și $y > Y$! Această observație ne scapă de necesitatea unei structuri de date tridimensionale.

Algoritmul

Pentru calculul eficient vom folosi **Arbori Indexați Binar** pentru a menține suma valorilor și numărul de puncte de comisie.

1. **Calculul pentru S_{total}, S_x și S_y:** Sortăm toate evenimentele (atât update-urile, cât și query-urile) descrescător după valoarea lor principală (V_A pentru update-uri, V_Q pentru query-uri). Parcurgem evenimentele:
   - Când întâlnim un update, îl adăugăm într-un contor global și actualizăm două AIB-uri (unul pentru coordonata x, altul pentru y).
   - Când întâlnim un query, S_{total} se calculează din totalul acumulat. S_x și S_y se calculează interogând cele două AIB-uri pentru valori strict mai mari decât X, respectiv Y.
2. **Calculul pentru S_{xy}:** Din moment ce restricția $V_A > V_Q$ pică, trebuie să respectăm doar $x > X$ cu $y > Y$. Sortăm evenimentele descrescător doar după coordonata x. Parcurgem lista, introducând update-urile într-un AIB nou după coordonata y. Când întâlnim un query, aflăm valoarea S_{xy} interogând acest AIB pentru update-urile cu coordonata strict mai mare ca Y.

Acest procedeu ne oferă inspirația pentru cadranul stânga-jos. Pentru a calcula contribuțiile celorlalte 3 cadrane (dreapta-jos, stânga-sus, dreapta-sus), aplicăm exact același algoritm reflectând planul (înmulțind axa x, axa y sau ambele cu -1).

Complexitatea finală de timp este $O((N+Q)\log(N+Q))$, dominată de sortări și actualizările din structurile de date AIB. Complexitatea spațială este $O(N + Q)$.

Problema 2. Influencer

Propusă de: Stud. Alexandru Lorintz, Universitatea "Babeş-Bolyai", Cluj-Napoca

Pentru a rezolva această problemă, vom construi intuiția pas cu pas, pornind de la cazurile de bază ($M = 0$ și $M = 1$) pentru a înțelege cum legăturile speciale afectează conectivitatea rețelei.

Subtask-ul $M = 0$ (Fără legături speciale)

Fără nicio coardă, avem doar un cerc format din N utilizatori. Ne dorim ca acest cerc să fie tare conex folosind doar muchiile conturului, configurate cu L, R, B sau N.

Să analizăm numărul de muchii de tip N (lipsa conexiunii) pe care le putem avea:

- Dacă avem **două sau mai multe** muchii de tip N, rețeaua se rupe în bucăți disjuncte, fiind imposibil ca informația să ajungă peste tot.
- Dacă avem **exact o** muchie de tip N, cercul devine un simplu lanț deschis. Singura modalitate ca un lanț să fie tare conex este ca absolut toate celelalte $N - 1$ muchii să fie bidirecționale (B). Deoarece muchia N poate fi plasată în oricare dintre cele N poziții, avem N astfel de configurații valide.
- Dacă **nu avem nicio** muchie de tip N, fluxul pe cerc trebuie să formeze cel puțin un ciclu direcționat. Nu putem amesteca muchii stricte L cu muchii stricte R (fără a fi perfect "acoperite" de B-uri), deoarece am crea puncte de acumulare sau epuizare a informației (noduri în care doar intră sau din care doar iese flux). Așadar, ori folosim doar L și B (2^N moduri), ori doar R și B (2^N moduri). Cum cazul în care toate muchiile sunt B a fost numărat de două ori, numărul de configurații pentru această situație este $2^N + 2^N - 1 = 2^{N+1} - 1$.

Însumând aceste cazuri, numărul total de configurații pentru un arc de lungime N fără alte constrângeri interne este dictat de funcția:

$$f(N) = 2^{N+1} + N - 1$$

Subtask-ul $M = 1$ (O legătură specială bidirecțională)

Ce se întâmplă când adăugăm o singură coardă între nodurile u și v? Coarda (fiind o legătură specială bidirecțională) împarte cercul în două arce de lungimi K_1 și K_2 (unde $K_1 + K_2 = N$).

Observația fundamentală a problemei este că aceste două arce devin complet **independente**. Pentru ca rețeaua să fie tare conexă, este suficient și necesar ca nodurile din primul arc să se poată conecta la "nucleul" format de capetele corzii $\{u, v\}$, și la fel pentru nodurile din al doilea arc. Orice mesaj între un nod dintr-o parte și unul din cealaltă parte va trece garantat prin u și v.

Cum numărăm modurile de a configura unul dintre aceste arce, să spunem de lungime K? Situația este exact echivalentă cu un ciclu izolat în care coarda joacă rolul unei muchii B sigure, garantate. Astfel, trebuie doar să decidem orientările celor K muchii de pe contur astfel încât,

împreună cu coarda, să formeze o componentă tare conexă. Raționamentul de la Subtask-ul $M = 0$ se aplică identic: formula este $f(K)$. Prin urmare, răspunsul pentru $M = 1$ va fi pur și simplu produsul $f(K_1) \cdot f(K_2)$.

Soluția completă (Pentru orice M)

Proprietatea de independență demonstrată la $M = 1$ se aplică pentru oricâte corzi care nu se intersectează (un graf outerplanar). Orice set de corzi împarte cercul inițial în multiple segmente de contur complet independente, delimitate de nodurile care fac parte din "scheletul" de corzi.

Problema se reduce astfel la identificarea lungimilor tuturor acestor fragmente izolate de contur, să le notăm $L_1, L_2, \ldots, L_P$. Răspunsul final va fi produsul tuturor $f(L_i)$ modulo $10^9 + 7$.

Pentru a extrage aceste segmente în complexitate optimă de $O(N + M)$, soluția folosește o parcurgere elegantă a conturului simulată printr-o stivă:

1. Construim o listă de adiacență pentru corzi, păstrând pentru fiecare nod doar muchiile către noduri cu indici mai mari (creând un graf orientat pe indici).

2. Utilizăm o stivă care reține starea procesării unui arc sub forma tripletului `(stânga, dreapta, lungime_curentă)`.

3. Parcurgând graful, atunci când întâlnim un nod care este originea unei corzi către un alt nod x facem o "tăietură": adăugăm în stivă noul sub-interval creat de coardă, împărțind logic poligonul mare în poligoane mai mici.

4. Când un sub-interval a fost complet închis (procesarea ajunge la capătul său drept), lungimea acumulată pe conturul său direct len ne oferă un bloc independent. Înmulțim răspunsul global cu valoarea $f(len)$ (modulo $10^9 + 7$) și continuăm rezolvarea pentru restul figurii.

Această abordare izolează conturul efectiv de deasupra fiecărei corzi, ignorând eficient spațiile interioare complet delimitate de corzi (pentru care lungimea conturului devine 0, iar $f(0) = 1$, fiind deci element neutru la înmulțire). Complexitatea de timp și spațiu este liniară.

Problema 3. Aquilla

Propusă de: Stud. Alexandru Lorintz, Universitatea "Babeş-Bolyai", Cluj-Napoca

Fie $DP[i]$ suma produselor tuturor subșirurilor *șmechere* care se termină la indicele i. Din condiția problemei, deducem relația de recurență de bază:

$$DP[i] = v_i \cdot \left(1 + \sum_{\substack{j<i \\ \min(d_i, d_j) \mid (i-j)}} DP[j] \right) \pmod{10^9 + 7}$$

Răspunsul final va fi $\sum_{i=1}^{N} DP[i] \pmod{10^9+7}$. Calculul direct al acestei recurențe necesită un timp de execuție de $O(N^2)$, suficient doar pentru obținerea unui punctaj parțial (subtask-urile cu

$N \leq 2000$). Pentru a obține soluția optimă de 100 de puncte, vom aplica tehnica descompunerii în radical (Square Root Decomposition). Alegem o constantă $B \approx \sqrt{N}$ (de exemplu, $B = 450$).

Când ne aflăm la pasul i și dorim să calculăm suma tranzițiilor valide, clasificăm elementele anterioare $j < i$ în două cazuri principale, în funcție de minimul dintre d_i și d_j, iar apoi rafinăm în funcție de valoarea pragului B.

Cazul 1: $\min(d_i, d_j) = d_i$ (implică $d_i < d_j$)

Aici condiția de divizibilitate devine $d_i \mid (i - j)$, ceea ce înseamnă că salturile se pot face precis, din d_i în d_i înapoi de la i.

- **Subcazul 1.1:** $d_i \geq B$. Numărul de pași multipli de d_i către stânga este de cel mult $\frac{N}{B} \approx \sqrt{N}$. Astfel, putem itera brut prin indicii $j = i - d_i, i - 2 \cdot d_i, \ldots$ și să adăugăm $DP[j]$ la totalul curent doar dacă respectă condiția de minim ($d_j > d_i$). Complexitate: $O(\sqrt{N})$.

- **Subcazul 1.2:** $d_i < B$. Când d_i este mic, iterarea brută este prea lentă. Folosim o structură de date suplimentară (o matrice `sumB[m][r]`), în care menținem proactiv suma valorilor $DP[k]$ pentru toate elementele procesate până în prezent care respectă $d_k \geq m$ și $k \equiv r \pmod{m}$. Pentru a afla rezultatul, interogăm direct `sumB[`d_i`][`$i \pmod{d_i}$`]` în $O(1)$. Actualizarea acestei structuri se face la finalul calculării fiecărui $DP[i]$: adunăm valoarea tocmai calculată în `sumB[m][`$i \pmod{m}$`]` pentru toți $m \leq \min(d_i, B - 1)$. Acest pas necesită $O(B)$ per element i.

Cazul 2: $\min(d_i, d_j) = d_j$ (implică $d_j \leq d_i$)

În acest scenariu, condiția devine $d_j \mid (i - j)$, ceea ce înseamnă că elementele anterioare j sunt cele care „dictează” pașii către i.

- **Subcazul 2.1:** $d_j < B$. Reținem o a doua matrice de stări, `sumSmall[m][r]`, care stochează exact suma valorilor $DP[k]$ pentru elementele cu $d_k = m$ și $k \equiv r \pmod{m}$. Pentru a aduna contribuția acestora, iterăm prin toate valorile posibile ale lui $m \in [1, \min(d_i, B) - 1]$ și adunăm `sumSmall[m][`$i \pmod{m}$`]`. Complexitatea interogării este mărginită de $O(B)$. Actualizarea structurii este însă în $O(1)$: dacă la finalul pasului i observăm că $d_i < B$, pur și simplu adunăm $DP[i]$ în clasa sa de resturi `sumSmall[`d_i`][`$i \pmod{d_i}$`]`.

- **Subcazul 2.2:** $d_j \geq B$. Acest caz prespune implicit că și $d_i \geq B$. Observăm că un element anterior j cu un prag d_j mare are foarte puțini multipli (cel mult $\sqrt{N}$ pe întreg vectorul). Prin urmare, lăsăm indicele j să își propage singur valoarea către viitor (o abordare de tip *lazy forward propagation*). Menținem liste cu *actualizări active* (`activeUpdates`) pentru fiecare index. Când algoritmul ajunge la poziția i, iterăm prin actualizările active programate pentru indexul i. Dacă pasul actualizării (d_j) este $\leq d_i$, o cumulăm în starea noastră. Pentru a menține eleganța și un consum de memorie de $O(N)$, după ce procesăm actualizarea în i, o reprogramăm prin mutarea ei la poziția $i + d_j$ (doar dacă aceasta nu depășește N).

Asamblând aceste cazuri, algoritmul calculează complet tabloul DP combinând interogări eficiente din stări precalculate (pentru praguri mici) și propagări active limitate (pentru praguri mari). Complexitatea finală a soluției se reduce astfel la $O(N\sqrt{N})$ timp de execuție, utilizând $O(N + B^2)$ memorie auxiliară.

Echipa

Problemele pentru acest baraj au fost pregătite de:

- Tamio-Vesa Nakajima, Universitatea Marburg, Hesse
- Andrei-Costin Constantinescu, ETH, Zürich, Elveția
- Adrian Panaete, Colegiul Național "A.T. Laurian", Botoșani
- Zoltan Szabó, Inspectoratul Școlar Județean, Târgu-Mureș
- Mihai Bunget, Colegiul Naţional "Tudor Vladimirescu", Târgu Jiu
- Alexandru Ioniță, Universitatea Alexandru Ioan Cuza, Iaşi
- Paul Diac, Universitatea Alexandru Ioan Cuza, Iaşi
- Alexandru Lorintz, Universitatea "Babeş-Bolyai", Cluj-Napoca
- Toma Ariciu, Universitatea Politehnică Bucureşti
- Andrei-Cristian Ivan, Universitatea Politehnică Bucureşti
- Dan-Constantin Spătărel, București
- Vlad-Mihai Bogdan, Universitatea București
- Adrian-Andrei Roșu, Universitatea Politehnică Bucureşti
- Lucian-Andrei Badea, TU Delft, Olanda
- Eugenie-Daniel Posdărăscu, Youni, București
- Radu-Teodor Prosie, Universitatea București
- Liviu Armand Gheorghe, Universitatea Bucureşti
- Andrei-Cristian Turcu, Universitatea București
- Tudor Iacob, Universitatea "Babeş-Bolyai", Cluj-Napoca
- Alexandru Peticaru, Universitatea Politehnică Bucureşti
- Tiberiu Cozma, Universitatea Alexandru Ioan Cuza, Iaşi

- Cristian-Adrian Blaga, Universitatea ”Babeş-Bolyai”, Cluj-Napoca
- Andrei Leţu, Universitatea ”Babeș-Bolyai”, Facultatea de Matematică și Informatică, Cluj-Napoca
- Alex-Matei Ignat, Universitatea din București, Facultatea de Matematică și Informatică, București
- Mihai Ciucu, CS Academy, București
- Alexandra Nicola, Google, București
- Luca Mihai Ilie, Ecole Polytechnique, Paris
- Matei-Octavian Stănescu, Universitatea Națională de Știință și Tehnologie Politehnica, București